Beyond Boolean Intersections: A Type-Safe Verified Bisimulation Between D-Separation Trails and Moral-Graph Reachability

ANONYMOUS AUTHOR(S)

D-separation is the central graph-theoretic criterion for reading conditional independencies off a directed acyclic graph. We mechanize its two standard characterizations—trail blocking [**?**] and moralized ancestral-graph separation [**?**]—as a *verified bisimulation* between an operational semantics of active trails and a static analysis of moralized-graph reachability. Our formalization is organized in three layers: a type-safe graph syntax whose pairwise-disjointness invariant (DisjointSets X Y Z) acts as an affine ownership constraint; a type-indexed intermediate representation (MAGWalk, StaticRoute, OpenTrace, ActiveRoute); and constructive witness-extraction pipelines in both directions.

The forward direction is a *certified trace optimizer*: it compiles any active trail through a Bayes-ball state machine into a compressed moral-graph walk, certifying soundness (moral-graph separation $\Rightarrow$ trail blocking) under the exact domain restriction that repairs equivalence. The reverse direction is a *constructive decompiler of semantic flow*: from moral-graph reachability we deterministically reconstruct an explicit active-trail witness, including a proved bad-collider normalization that strictly decreases a well-founded measure. Both directions compose into a closed equivalence on the pairwise-disjoint domain (`dSeparated_iff_dSeparates`); the reverse pipeline carries no proof debt.

We also machine-check the first mechanized counterexample to unrestricted equivalence, showing that the endpoint treatment is the sole culprit. All graph-semantic results are constructive and verified in Lean 4 with no **`sorry`**. We position this library as a graph-semantic substrate for quantitative information-flow bounds; the bridge to probabilistic conditional independence is stated as an explicit scaffold and is *not* discharged in this artifact.

1 Introduction

Consider the simplest possible directed acyclic graph: a chain of three nodes $0 \longrightarrow 1 \longrightarrow 2$. Let $X = \{0\}$, $Y = \{1\}$, and $Z = \{0\}$. Does Z d-separate X from Y?

The two standard textbooks give different answers:

- **Trail blocking** [**?**] says: examine every undirected trail from X to Y; if every internal triple is blocked by Z, the sets are d-separated. The one-edge trail $0 \longrightarrow 1$ has no internal triple, so it is not blocked. Hence X and Y are *not* d-separated.
- **Moral graph separation** [**?**] says: take the ancestral subgraph of $X \cup Y \cup Z$, moralize it, delete Z, and check whether X and Y are disconnected. Node 0 is in Z, so it is deleted. The remaining graph has no path from 0 to 1, so X and Y *are* d-separated.

We mechanize this discrepancy and its repair, treating the two characterizations not as interchangeable definitions but as a *verified bisimulation* between two semantic layers:

- an **operational semantics** of active trails (the "execution" layer of information flow);
- a **static analysis** of moralized-graph reachability (the "compiled" layer).

The forward direction (soundness) is a *certified trace optimizer* that compiles an operational trace into static reachability evidence. The reverse direction (completeness) is a *constructive decompiler of semantic flow*: it reconstructs a concrete execution witness—an exploitability certificate for the information leak—from static reachability alone. The counterexample says exactly that the two languages are *not* bisimilar without a well-typed domain restriction.

1.1 Contributions

Our contributions are:

(1) We formalize both d-separation characterizations in Lean 4 over finite DAGs with natural-number nodes, organized as a three-layer stack: type-safe graph syntax, certified intermediate representations, and constructive witness-extraction pipelines.
(2) We construct and machine-check a concrete counterexample, proving that unrestricted equivalence is false and identifying the exact repair (DisjointSets X Y Z).
(3) We mechanize the **forward bisimulation** (soundness): an explicit compilation pipeline Trail → BayesBallPath → MAGWalk → Reachable, certified by node-survival proofs.
(4) We mechanize the **reverse bisimulation** (witness-producing completeness): a decompilation pipeline Reachable → StaticRoute → OpenTrace → ActiveRoute → ∃ Trail, ¬isBlocked. Every stage is type-indexed and constructive, including a proved bad-collider normalization (`exists_split`, `ancestor_escape`, `route_improves_of_bad`) that strictly decreases a well-founded measure. The reverse pipeline carries no **`sorry`**.
(5) We close the **full equivalence** (`dSeparated_iff_dSeparates`): on the pairwise-disjoint domain, moral-graph separation holds if and only if every trail is blocked, with both directions discharged by the optimizer and the decompiler.
(6) We delimit the boundary to **quantitative information flow**: the bridge from d-separation to probabilistic conditional independence is given as an explicit Lean scaffold (`dSeparation_implies_conditional_independence`) and is deliberately left undischarged, marking precisely where graph semantics ends and measure-theoretic work begins.

All results are available as a standalone Lean 4 package. The decompiler produces not a mere existence proof but a computational witness—a Σ-type containing the actual trail and the proof that it is unblocked.

2 A Three-Layer Architecture

Our formal development is organized in three layers that mirror the architecture of a verified compiler.

2.1 Layer 1: Type-Safe Graph Syntax

Nodes are natural numbers, edges are a finite set of ordered pairs (Finset ($\mathbb{N} \times \mathbb{N}$)), and acyclicity is well-foundedness of the edge relation. The key safety invariant is `DisjointSets X Y Z`, which requires pairwise disjointness of the three node sets. In the language of substructural types, this is an *affine ownership constraint*: the conditioned variables Z are borrowed by the query and cannot alias the source X or sink Y. For the extrinsic presentation, we target a first-order surface calculus with an explicit environment and no higher-order binders, so the metatheory stays focused on queries rather than substitution-heavy closure machinery. Surface graphs carry a rank certificate for acyclicity, checked locally and then elaborated into intrinsic DAGs via `DAG.ofRank`.

Listing 1. DAG structure and safety predicate

```
structure DAG where
  nodes : Finset \N
  edges : Finset (\N \times \N)
  edges_subset : \forall {u v}, (u,v) \in edges -> u \in nodes \wedge v \in nodes
  acyclic : WellFounded fun u v => (u,v) \in edges

def DisjointSets (X Y Z : Finset \N) : Prop :=
  Disjoint X Y \wedge Disjoint X Z \wedge Disjoint Y Z
```

2.2 Layer 2: Certified Intermediate Representations

Between operational trails and graph reachability sits a family of IRs that preserve just enough structure for certified translation.

MAGWalk. A compressed walk language equivalent to moral-graph reachability. It has three constructors: reflexivity, a single directed or reversed edge (`MAGWalk.single`), and a "moral jump" over an active collider (`MAGWalk.jump`). The collider node is skipped, which is why compression is correct.

StaticRoute. A type-valued decompiler IR that keeps the witness hidden by graph reachability. Each step is either a forward edge, a backward edge, or a moral jump storing the common child:

Listing 2. Static route IR (excerpt)

```
inductive StaticStep (G : DAG) (X Y Z : Finset \N) : \N -> \N -> Type where
  | directForward (hEdge : G.HasEdge u v) ...
  | directBackward (hEdge : G.HasEdge v u) ...
  | moralJump (huw : G.HasEdge u w) (hvw : G.HasEdge v w)
      (hw : w \in G.ancestralSubgraphNodes (X \cup Y \cup Z)) ...
```

OpenTrace. The compiler target before converting to an active route. Each `cons` node stores its oriented DAG traversal *and* a local non-blocking proof, making `OpenTrace` a type-valued certificate that every junction is Z-open.

2.3 Layer 3: Witness-Extraction Pipelines

The two pipelines are summarized in Figure 1.

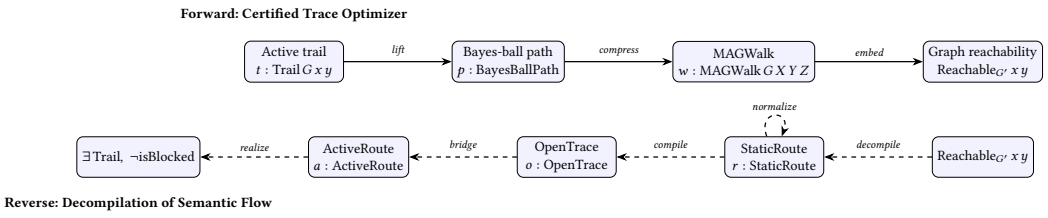


Fig. 1. The two witness-extraction pipelines. Solid arrows: forward (soundness). Dashed arrows: reverse (completeness). Every arrow, including the *normalize* step, is a total constructive function in Lean; the normalize step is a proved bad-collider minimization (Section 6).

3 Background: The Two Criteria

3.1 Trail Blocking (Operational Semantics)

An undirected *trail* from x to y in a DAG G is a finite walk that may follow edges forward or backward. A consecutive triple $a \to b \to c$ or $a \leftarrow b \to c$ is a *non-collider*; $a \to b \leftarrow c$ is a *collider*. A triple is *blocked* by Z if:

- it is a non-collider whose middle node is in Z; or
- it is a collider whose middle node and all descendants are disjoint from Z.

A trail is *blocked* by Z if some internal triple is blocked. Then Z *d-separates* X from Y in the trail sense if every trail from X to Y is blocked by Z.

3.2 Moral Graph Separation (Static Analysis)

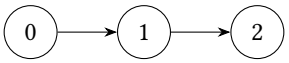
The *ancestral subgraph* of S is the subgraph induced by all ancestors of nodes in S. Its *moral graph* is obtained by (1) forgetting edge directions and (2) connecting all co-parents (pairs of nodes with a common child). The *d-separation graph* is the moral graph of the ancestral subgraph of $X \cup Y \cup Z$ with Z removed. Then Z d-separates X from Y in the graph sense if no node of X is connected to any node of Y in this graph.

3.3 Textbook Equivalence and Its Caveat

Standard references state that the two criteria are equivalent under the implicit assumption that X, Y, and Z are pairwise disjoint. In applications, however, the disjointness precondition is often omitted. Our counterexample (Section 4) shows that this omission is not benign.

4 The Endpoint Caveat: A Counterexample

Our counterexample lives on the simplest non-trivial DAG:



Let $X = \{0\}$, $Y = \{1\}$, $Z = \{0\}$.

Theorem 4.1 (Endpoint-in-Z counterexample). *There exists a DAG G and finite sets $X, Y, Z \subseteq \mathbb{N}$ such that G d-separates X from Y in the moral-graph sense, but Z does* not *d-separate X from Y in the trail-blocking sense.*

Listing 3. Mechanized counterexample

```
theorem dsep_complete_endpoint_in_Z_counterexample :
   \exists (G : DAG) (X Y Z : Finset \N),
     DAG.dSeparated G X Y Z \wedge \neg dSeparates G X Y Z := by
 refine <chain3, {0}, {1}, {0}, ?_, ?_>
 ...
```

Why the disagreement happens. Trail blocking only inspects *internal* triples. The one-edge trail $0 \longrightarrow 1$ has no internal triple at all, so nothing can block it. By contrast, the moral-graph construction deletes Z unconditionally; since $0 \in Z$, the edge $(0, 1)$ disappears from the moralized graph, and 0 and 1 are disconnected.

The standard repair. If we require $X \cap Z = \emptyset$ and $Y \cap Z = \emptyset$, the start node of any trail is never deleted, and the endpoint caveat disappears.

5 Forward Bisimulation: The Certified Trace Optimizer

Under DisjointSets X Y Z, we mechanize the soundness direction:

$$\text{DAG.dSeparated}(G, X, Y, Z) \implies \text{dSeparates}(G, X, Y, Z).$$

The proof follows a three-stage compilation pipeline.

Stage 1: Trail $\longrightarrow$ Bayes-ball path. If a trail t from $x \in X$ to $y \in Y$ is not blocked by Z, and $x \notin Z$ (guaranteed by Disjoint X Z), then we lift t to an explicit Bayes-ball state path. The construction is by induction on the trail, analyzing each directional triple to show that the Bayes-ball state machine can traverse it.

Listing 4. Trail to Bayes-ball path

```
def bayesBallPath_of_active_trail_outOf
    (t : Trail G u v) (h_active : \neg t.isBlocked Z)
    (huZ : u \notin Z) :
    \Sigma final_dir, BayesBallPath G Z (u, outOf) (v, final_dir)
```

Stage 2: Certified membership. The Bayes-ball path contains intermediate states that must "survive" deletion of Z. We prove a certified version that guarantees every `RequiredState` node belongs to the d-separation graph nodes:

Listing 5. Certified Bayes-ball path

```
def bayesBallPathCert_of_active_trail_outOf
    (t : Trail G u v) (h_active : \neg t.isBlocked Z)
    (huZ : u \notin Z)
    (huA : u \in G.ancestralSubgraphNodes (X \cup Y \cup Z))
    (hvD : v \in G.dSeparationGraphNodes X Y Z) :
    \Sigma final_dir, {p : BayesBallPath ... //
      \forall {q}, BayesBallPath.RequiredState p q ->
        q.1 \in G.dSeparationGraphNodes X Y Z}
```

Stage 3: Compress to MAGWalk. A Bayes-ball path is a sequence of single-edge steps. The `compress` algorithm scans it with a two-step window: ordinary non-collider windows become a `MAGWalk.single`, while active-collider windows $(a, \_) \to (b, \text{into}) \to (c, \text{outOf})$ become a single `MAGWalk.jump` over the collider b. The collider itself need not survive deletion of Z, which is why this compression is correct.

Theorem 5.1 (Soundness). *For any DAG G and pairwise-disjoint finite sets $X, Y, Z \subseteq \mathbb{N}$, if G d-separates X from Y in the moral-graph sense, then every trail from X to Y is blocked by Z.*

Listing 6. Top-level soundness theorem

```
theorem dSeparated_of_dSeparated_disjoint
    (hXYZ : DisjointSets X Y Z)
    (hsep : DAG.dSeparated G X Y Z) : dSeparates G X Y Z
```

6 Reverse Bisimulation: Decompilation of Semantic Flow

The reverse direction is the decompiler: from ¬DAG.dSeparated it constructively reconstructs an active-trail witness—an exploitability certificate showing that the conditioning set Z fails to block the flow. The pipeline is:

$$\begin{aligned}\text{Reachable} &\to \text{StaticRoute} \to \text{NormalizedStaticRoute} \\ &\to \text{OpenTrace} \to \text{ActiveRoute} \to \exists\,\text{Trail},\ \neg\text{isBlocked}.\end{aligned}$$

6.1 From Reachability to StaticRoute

Graph reachability is a `Prop`-valued relation that hides the structure of each step. We introduce `StaticStep` and `StaticRoute` as type-valued evidence: a `StaticRoute` is an explicit list of directed edges, reversed edges, and moral jumps, each carrying the witness needed for later stages.

Listing 7. Reachability to static route

```
theorem nonemptyStaticRoute_of_dSeparationGraph_reachable
```

```
    (h : (G.dSeparationGraph X Y Z).Reachable u v) :
    Nonempty (StaticRoute G X Y Z u v)
```

6.2 Normalization: Eliminating Bad Colliders

A "bad collider" is a collider window whose middle node and all descendants are disjoint from Z. Such a collider is blocked, so it must be eliminated. Normalization may change endpoints: if a bad collider leaks ancestry into X or Y, we reroute the prefix or suffix to a different node in that set.

We package static routes as endpoint-carrying witnesses (`StaticRouteWitness`) and define a bad-collider count `routeBadCount`. The minimization is a `Nat.find` wrapper: if every nonzero-count witness can be strictly improved, a zero-count witness exists.

Listing 8. Bad-collider minimization wrapper

```
theorem normalized_route_exists_of_improves
    (hwit : \exists w : StaticRouteWitness G X Y Z, True)
    (himprove : \forall w, routeBadCount w \neq 0 ->
      \exists w', routeBadCount w' < routeBadCount w) :
    \exists w, routeBadCount w = 0
```

The improvement obligation is discharged by the core normalization theorem, which is fully proved (no `sorry`):

Listing 9. Strict improvement for non-normalized routes

```
theorem route_improves_of_bad
    (w : StaticRouteWitness G X Y Z) (hbad : routeBadCount w \neq 0) :
    \exists w' : StaticRouteWitness G X Y Z,
      routeBadCount w' < routeBadCount w
```

The proof is a short dispatcher over three established lemmas. `exists_split` extracts the first bad collider as a `Split` carrying a zero-bad prefix and a count bound. `ancestor_escape` shows the bad child's leaked ancestry must reach a node in X or Y (its descendant cone misses Z, so it cannot dead-end). `bad_child_survives` and `escape_path_survives` prove that the rerouted escape chain stays inside the d-separation graph nodes. Reattaching the escape chain—backward into X or forward into Y—removes a moral jump of cost 1 and adds chains of cost 0; the strict decrease then follows by `countBadColliders_backwardEscape_append_suffix_lt` (resp. `countBadColliders_prefix_append_forwardEscape_lt`) and `omega`. As a deliberate design choice, the witness endpoints may change: rerouting is allowed to land on a *different* node of X or Y, which is what makes the escape construction total.

6.3 From Zero-Bad StaticRoute to OpenTrace to ActiveRoute

A static route with zero bad colliders compiles directly to an `OpenTrace`, because every junction is locally open:

Listing 10. Zero-bad compiler

```
theorem openTrace_of_countBadColliders_zero
    (hzero : countBadColliders arrival route = 0) :
    Nonempty (\Sigma finalDir, OpenTrace G Z (x, arrival) (y, finalDir))
```

The `OpenTrace` is then converted to a `BayesBallPath` and wrapped as an `ActiveRoute`. Finally, `ActiveRoute.to_activeTrail` yields the existential witness:

Listing 11. Final decompiler theorem

```
theorem activeWitness_of_not_dSeparated
   (hnot : \neg DAG.dSeparated G X Y Z) : ActiveWitness G X Y Z
```

The result is a constructive Σ-type containing the actual nodes $x \in X$, $y \in Y$, the final direction, and the active route.

6.4 The Closed Equivalence

Composing the optimizer (Section 5) and the decompiler yields the full equivalence on the well-typed domain—the bisimulation statement: on pairwise-disjoint X, Y, Z, the operational and static languages agree exactly.

Listing 12. Full d-separation equivalence (no proof debt)

```
theorem dSeparated_iff_dSeparates
   (hXYZ : DisjointSets X Y Z) :
   DAG.dSeparated G X Y Z <-> dSeparates G X Y Z
```

Remark 1 (Verification status). *The full Lean development typechecks (*`lake build`*). The graph-semantic core—both directions of* `dSeparated_iff_dSeparates` *and the entire reverse synthesis stack—contains no* `sorry`*. The only* `sorry`*s in the package are two* intentional *scaffold stubs in* `InfoTheoryBridge.lean` *(Section 8), which are out of scope for the equivalence claims.*

7 The PL Dictionary: Reframing Causal Graphs

To situate this work for a POPL audience, we reframe traditional causal-graph vocabulary in programming-languages terms:

Causal Inference	PL Framing
DAG	Dataflow AST / Dependency Graph Syntax
DisjointSets X Y Z	Affine ownership constraint (no aliasing of Z)
Active Trail	Operational trace of semantics
MAGWalk	Reachability abstract IR (compressed moral-graph walk)
Collider Jump (`MAGWalk.jump`)	Peephole optimization / semantic fold
Trail → BayesBallPath	Typestate tracking
BayesBallPath → MAGWalk	Certified trace optimizer
StaticRoute → OpenTrace → ActiveRoute	Semantic-flow decompiler / exploitability-certificate generator
Cut-Set / Mutual Information	Quantitative Information Flow (QIF) capacity bounds

Table 1. Reframing causal-graph concepts for a POPL audience.

8 Related Work

Our mechanization sits at the intersection of causal inference, mechanized probabilistic semantics, and verified compilation. We briefly survey the most closely related lines of work.

Mechanized Causal Inference and D-Separation. Formalizing the graph-theoretic foundations of causal inference is an active frontier. Recently, Chancelier, De Lara, and Heymann (2024) [**?**] mechanized d-separation as a binary relation using Coq and MathComp/SSReflect, providing a relational-algebraic treatment of graphical separation. Parallel to our work, Anna Zhang (2025) [**?**]

formalized the semantics of conditional independence and d-separation in Coq, bridging structural properties with function-based definitions for experimental design.

Both developments adopt standard mathematical formulations without exposing the implicit textbook domain caveats. Our work contributes the first formalized counterexample and repair (the endpoint caveat) for the gap between syntactic trails and moralized node-deletion. We also shift from purely relational verification to a constructive bisimulation that generates executable IRs (e.g., `OpenTrace`), optimized in Lean 4 for computational witness synthesis.

Categorical and Topological D-Separation. Fritz and Klingler (2023) [**?**], extending work by Fritz and Liang (2023) [**?**], abstracted d-separation using string diagrams in Markov categories. Their framework bypasses moralization, reading conditional independence directly from topological connectedness in a generalized measure-theoretic space. Our approach is complementary: Markov categories provide a general topological theory, while our artifact targets the legacy operational graph definitions (moral graphs and trace trails) implemented in existing software. By defining `DisjointSets` as an affine ownership type, we show that classical d-separation behaves like a resource-restricted syntax within a broader categorical structure.

Verified Trace Bisimulation and Witness Extraction. The forward pipeline (soundness) translates unbounded trace semantics into a compressed step-relation (`MAGWalk`), analogous to the semantics-preserving translations in Leroy's CompCert (2009) [**?**]. The reverse pipeline (completeness) deterministically unwinds reachability into a certified active trail, resembling Necula's Proof-Carrying Code (1997) [**?**]: instead of returning a boolean assertion of an information leak, our decompiler synthesizes an explicit, machine-checkable exploitability certificate. This provides a graph-theoretic substrate for future verified static analyzers of quantitative information flow.

8.1 Future Directions

Quantitative Information Flow (QIF).. Security analysis has moved from non-interference ("0 or 1") to measuring leak in bits. The verified bisimulation here is intended as the *graph-semantic substrate* for such bounds: a closed d-separation equivalence is what licenses replacing a conditional-independence assumption by a checked graph property in a cut-set capacity argument. Crossing from graph reachability to a probabilistic leak bound requires the bridge

$$\text{d-separation} \implies \text{conditional independence} \implies \text{conditional DPI} \implies \text{cut-set bound}.$$

We state the first step of this bridge as an explicit Lean scaffold, `dSeparation_implies_conditional_independence` in `InfoTheoryBridge.lean`, together with stubs for `FinitePMF` and `MarkovCompatible`. This scaffold is deliberately *not* discharged in this artifact: it carries two intentional **`sorry`**s and is the sole proof debt in the package. The finite-measure and information-theory machinery (entropy, conditional mutual information, conditional DPI, dual/KKT certificates) lives in a separate verification stack; integrating it with the DAG semantics formalized here—unifying the two DAG definitions and proving the conditional-independence step—is the principal item of future work.

9 Conclusion

We formalized the two standard d-separation criteria in Lean 4 as a verified bisimulation between an operational semantics of active trails and a static analysis of moralized-graph reachability. We machine-checked a concrete counterexample to unrestricted equivalence, characterized the repair (pairwise disjointness as an affine ownership constraint), and discharged both directions: a certified trace optimizer for soundness and a constructive, witness-producing decompiler—with a proved bad-collider normalization—for completeness. The two compose into the closed theorem `dSeparated_iff_dSeparates`, and the graph-semantic core carries no proof debt. What remains

is the deliberately delimited bridge to probabilistic conditional independence, stated here as a scaffold. Closing it would turn this graph-semantic substrate into machine-checked quantitative information-flow bounds.